

What kind of system is it?

Mathew Smith & Aneta Stefanovska

Consolidating Practical 1 and preparing for Practical 2



Where we are going today

Part A: recap of the methods

What you used in the practical, how the methods compare, and **when to use which**.

- Euler $\rightarrow$ RK2 $\rightarrow$ RK3 $\rightarrow$ RK4
- How to choose the time step
- How to check a numerical result

Part B: what kind of system?

Before you simulate or analyse data, form a **hypothesis** about what to expect.

- Isolated, closed, open
- Linear, nonlinear
- Static, dynamic; autonomous, nonautonomous
- Deterministic, stochastic

Part A tells you how to **compute**; Part B tells you what you should **expect**.

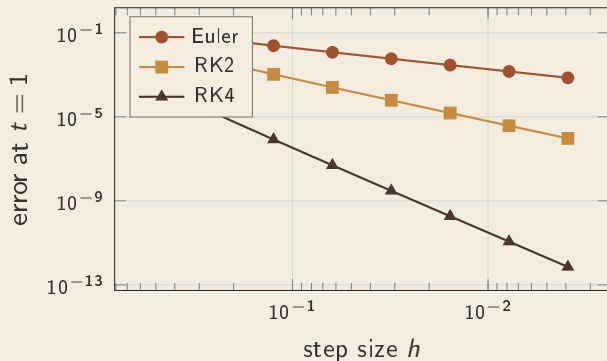
Time-stepping methods at a glance

Method	Cost per step	Global error	Use when
<i>Time stepping: solving $\dot{y} = f(t, y)$</i>			
Euler (RK1)	1 slope	$O(h)$	Understanding the idea; quick checks
RK2 (midpoint)	2 slopes	$O(h^2)$	A cheap upgrade; rarely the final choice
RK3	3 slopes	$O(h^3)$	A stepping stone to RK4
RK4	4 slopes	$O(h^4)$	Default for smooth, non-stiff problems

The pattern

Each extra slope evaluation per step buys one more order of accuracy. RK4 is the usual sweet spot; beyond it, the gain per extra evaluation shrinks.

What the methods do: error against step size



Test problem: $\dot{y} = -y$, $y(0) = 1$,
exact $y(1) = e^{-1}$.

On a log-log plot the error is a straight line, and its **slope is the order** of the method.

Read the plot

Same h , very different accuracy.
Smaller h helps every method, but
RK4 gains far faster.

Order of accuracy: your first check on any result

A method of **order** p has global error $\approx Ch^p$, with a constant C . Halving h divides the error by about 2^p .

When the exact answer is known

Test problem of slide 4, $\dot{y} = -y$, exact $y(1) = e^{-1}$.
Ratio $\text{error}(h)/\text{error}(h/2)$ for $h = 2^{-7} \rightarrow 2^{-8}$:

	Expected 2^p	Our runs
Euler	2	2.0
RK2	4	4.0
RK4	16	16.1

When it is not (the usual case)

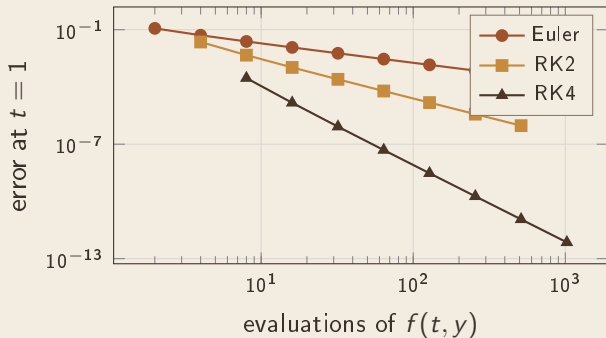
Run with h , $h/2$ and $h/4$ and compare **successive results**:

$$\frac{y_h - y_{h/2}}{y_{h/2} - y_{h/4}} \approx 2^p$$

No exact solution is needed.

Never trust a single run: if the ratio is not near 2^p , something is wrong.

Accuracy per unit of work



A **slope evaluation** is one computation of $f(t,y)$, the main cost of a run. RK4 needs four per step, but far fewer steps.

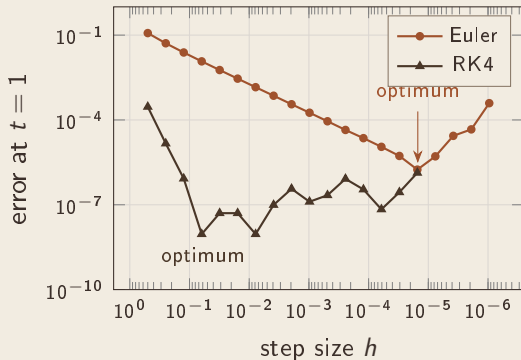
- RK4 with 128 evaluations:
error $\approx 3 \times 10^{-9}$
- Euler with 256 evaluations:
error $\approx 7 \times 10^{-4}$

Compare at equal work

The right question is not “cost per step” but “cost for a given accuracy”.

Choosing the time step

$\dot{y} = -y$ in single precision



Like a sampling interval, h must **resolve the fastest relevant time scale**. But:

- **too large:** errors accumulate, and beyond a limit the solution is unstable;
- **too small:** more work, and round-off grows, so the error **rises again**.

In practice

1. Start from the physics: h small enough to resolve the fastest relevant time scale.
2. Halve h until the result changes by less than you need, then stop.
3. Or let an adaptive solver (RK45) choose h .

When RK4 is not enough

Unknown dynamics

You do not know how small h must be. **Adaptive methods** (e.g. RK45) estimate the error at each step and adjust h automatically.

Stiff problems

Processes act on very different time scales, and an explicit method must take tiny steps to stay stable. **Implicit methods** (backward Euler, BDF) remain stable with much larger steps.

In practice

Library solvers (`scipy.integrate.solve_ivp`) offer these. Choose by the **problem**, not by habit — and always verify convergence: halve a fixed h , or tighten an adaptive tolerance.

Which method for which problem?

Your problem	First choice
Smooth ODE, moderate accuracy	RK4, with an $h/2$ check
Unknown time scales, or you need error control	Adaptive RK (e.g. RK45)
Very different time scales (stiff)	Implicit / stiff solver
Learning how a method works	Euler

Whatever you choose: verify convergence.

For a fixed-step method: halve h and compare.

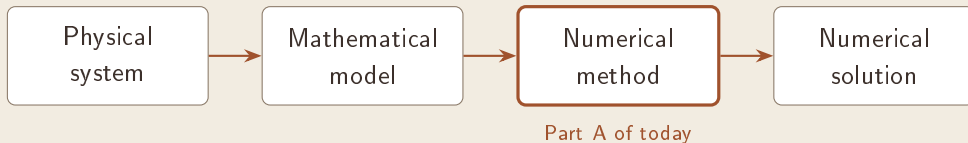
PART B

What kind of system is it?

Before running a simulation, or analysing data, ask what you should expect.

Isolated, closed or open · linear or nonlinear · static or dynamic, autonomous or not · deterministic or stochastic

From physical system to numerical solution



The methods you have just reviewed sit in the third box. Their **performance depends on the first two**.

The idea of Part B

Classify the system **before** you simulate. The classification tells you what the solution should look like, what to check, and which method from Part A is appropriate.

Four questions to ask of any system

Question	The answer is a label
1. What crosses the system boundary?	Isolated / Closed / Open
2. How do the variables interact?	Linear / Nonlinear
3. Does the state evolve in time, and does the rule itself change in time?	Static / Dynamic; Autonomous / Nonautonomous
4. Where, if anywhere, does randomness enter?	Deterministic / Stochastic

Each answer tells you **what to check** in a simulation and **which method** to use.

The labels are **independent**: one system can carry any combination. We take them in turn.

1. Classification by system boundary

The **system boundary** is the surface we draw around what we model; everything outside is the *surroundings*. The question is: what crosses it?

System	Crosses the boundary?		Example	What to check
	Mass	Energy		
Isolated	no	no	Two-body orbit	E , momentum conserved
Closed	no	yes	Piston–cylinder	Energy balance
Open	yes	yes	Stirred reactor (CSTR)	Mass and energy balances

Why this is the first question

The boundary decides which equations you need, and what to **check afterwards**: a conserved quantity (isolated) or a balance (closed, open).

Isolated system: the two-body orbit

Point masses, Newtonian gravity. **Boundary:** drawn around both bodies; no external force acts and nothing enters or leaves. With $\mu = G(m_1 + m_2)$ and $r = \sqrt{x^2 + y^2}$:

$$\begin{aligned}\dot{x} &= v_x, & \dot{y} &= v_y, \\ \dot{v}_x &= -\frac{\mu x}{r^3}, & \dot{v}_y &= -\frac{\mu y}{r^3}\end{aligned}$$

Start from $x_0 = r_0$, $y_0 = 0$, $v_{x0} = 0$, $v_{y0} = v_0$. The value of v_0 decides circular, elliptical or escape orbit.

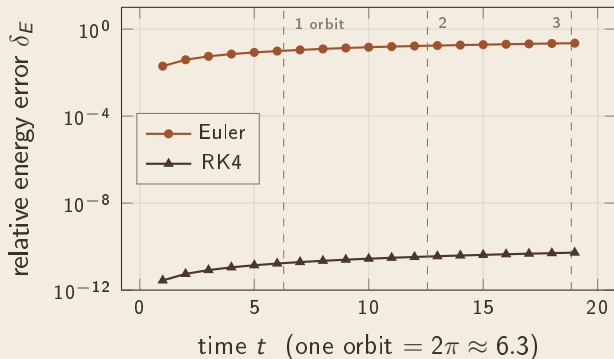
Hypothesis before simulating

Nothing crosses the boundary, so the **specific energy** E and **angular momentum** ℓ (per unit mass)

$$E = \frac{1}{2}v^2 - \frac{\mu}{r}, \quad \ell = xv_y - yv_x$$

should stay **constant**.

Does the numerical solution respect the physics?



Circular orbit, $\mu = 1$, $r_0 = 1$, $v_0 = 1$,
 $h = 0.01$; three orbits ($t \approx 19$).

$$\delta_E(t) = \frac{|E(t) - E(0)|}{|E(0)|}$$

Lesson

Euler's energy error grows to over 20% in three orbits; RK4's stays below 10^{-10} . A plausible-looking trajectory is **not** evidence of accuracy. Know what the energy *should* do before you read this curve.

Closed system: gas in a piston–cylinder

Boundary: the cylinder walls and the piston. Heat and work cross it; the gas does not.

$$\boxed{dU = \delta Q - \delta W}, \quad \delta W = P dV, \quad PV = nRT.$$

Hypothesis: slow compression at constant T

$n = 1$ mol at $T = 300$ K, volume halved. For an ideal gas U depends only on T , so $\Delta U = 0$:

$$Q = W = nRT \ln \frac{V_2}{V_1} \approx -1.73 \text{ kJ} :$$

the work done on the gas must leave as heat.

Check

Accumulate Q and W step by step along the process. At every step $U(t) - U(0) = Q - W$; here it must stay 0 to the accuracy of the step.

Open system: a stirred reactor (CSTR)

A **continuous stirred-tank reactor (CSTR)** is a well-mixed vessel with continuous inflow and outflow: the contents are uniform, so the outflow has the composition of the tank.

Reaction $A \rightarrow B$, first-order kinetics, flow Q through volume V :

$$\frac{dC_A}{dt} = \frac{Q}{V} (C_{A,\text{in}} - C_A) - kC_A$$

Hypothesis: set $dC_A/dt = 0$

$$C_{\text{ss}} = \frac{(Q/V) C_{A,\text{in}}}{Q/V + k}, \quad \tau = \frac{1}{Q/V + k}$$

C_A relaxes exponentially to C_{ss} with time constant τ .

A free test

This ODE is **linear**, so the exact solution is known: compare Euler and RK4 against it, and repeat the $h/2$ check.

2. Mathematical behaviour: linear systems

Mass–spring–damper, $m\ddot{x} + c\dot{x} + kx = F(t)$. With $x_1 = x$, $x_2 = \dot{x}$:

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = \frac{1}{m} [F(t) - cx_2 - kx_1].$$

What to expect

Superposition holds. With $c > 0$ free motion decays; if $c^2 < 4mk$ it oscillates at $\omega_d = \sqrt{k/m - (c/2m)^2}$. Periodic forcing gives a response at the forcing frequency.

How to check

Compare with the analytic solution, and read behaviour off the eigenvalues. RK4 is usually a safe default.

Nonlinear systems: the Van der Pol oscillator

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0$$

What to expect

For $|x| < 1$ the damping is negative and pumps energy in; for $|x| > 1$ it removes energy. The result is a **limit cycle**: a self-sustained oscillation to which trajectories converge from different initial conditions. Superposition fails.

Methods from Part A

For large μ the motion has fast and slow phases: expect **stiffness**. Use an adaptive or implicit solver, and always test several initial conditions.

Linear versus nonlinear

	Linear	Nonlinear
Superposition	Yes	Generally no
Exact solution	Often available	Often not
Typical behaviour	Decay, growth, response at forcing frequency	Limit cycles, several steady states, chaos
Numerical method	RK4 usually fine	May need adaptive or stiff solvers
Example	Mass–spring–damper	Van der Pol

The mathematical structure influences which method from Part A you should trust.

3. Static or dynamic? Autonomous or nonautonomous?

Static

$f(x) = 0$: an algebraic equation
⇒ root finding (Lecture 3)
e.g. the CSTR steady state

Dynamic

$\dot{x} = f(x, t)$: the state evolves
⇒ time stepping (Euler, RK4)

Autonomous

$\dot{x} = f(x)$: the rule is fixed in time
free oscillator, Van der Pol, CSTR

Nonautonomous (time-variable)

$\dot{x} = f(x, t)$: parameters or forcing vary in time
forced oscillator, CSTR with $C_{A,in}(t)$

What to expect

Autonomous: shifting the time origin changes nothing; steady states and limit cycles, if they exist, do not depend on time.

Nonautonomous: behaviour follows the time-varying parameters; a time-independent steady state need not exist; easily **mistaken for noise**.

4. Where does randomness enter?

Is the variation we see random, or do we just not know enough?

Time variability Parameters or forcing change in time (slide 21). Deterministic: **not** randomness.

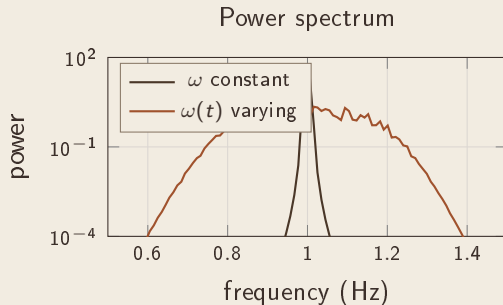
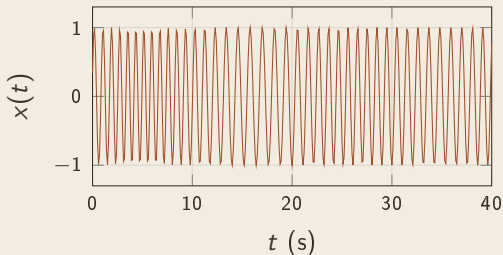
Uncertainty We do not know inputs, parameters or measurements exactly.

Stochasticity Randomness is part of the **dynamics**.

Related, but **not synonyms**.

Time variability can look like noise

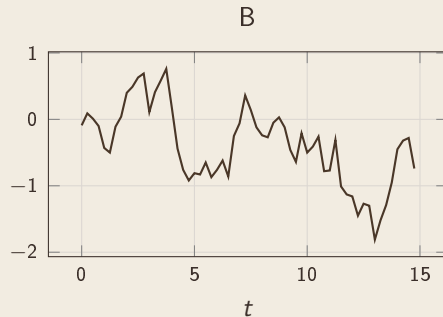
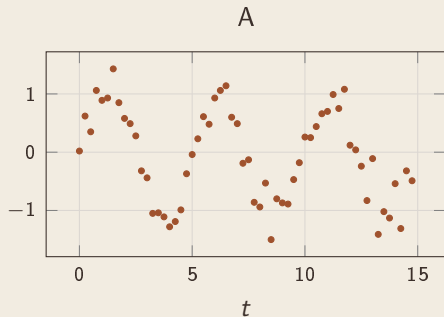
$$x = \sin \varphi(t), \quad \dot{\varphi} = \omega(t)$$



The same deterministic oscillator, once with constant frequency and once with $\omega(t)$ varying slowly in time. The sharp line spreads into a **broad band** that is easily read as noise.

Broad spectrum, or variance $> 0 \not\Rightarrow$ random

Which one is random?

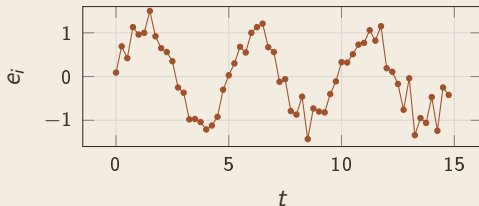


A is a deterministic oscillation plus measurement noise, $Y_i = X(t_i) + \varepsilon_i$. B is a random walk: randomness is in the **dynamics**. A histogram discards the **time ordering** that distinguishes them.

Look at the residuals

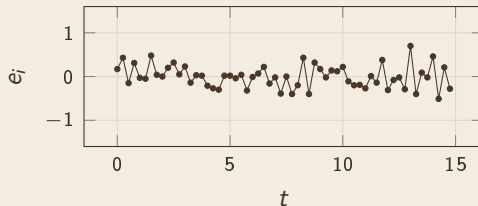
Fit a model $\hat{X}(t)$ to series A from the previous slide and look at the residuals $e_i = Y_i - \hat{X}(t_i)$ in time:

Model: a constant mean



Structured residuals: the model is **missing dynamics**, or its parameters vary in time.

Model: a sine



Noise-like residuals: only now describe them statistically, and say which kind of uncertainty they are.

Uncertain inputs, deterministic dynamics

$$\dot{x} = f(x, \theta, t), \quad \theta \sim p(\theta).$$

Monte Carlo: draw $\theta^{(1)}, \dots, \theta^{(N)}$, run the deterministic solver (e.g. RK4) for each, and study the ensemble:

$$\widehat{\mathbb{E}}[X(t)], \quad \widehat{\text{Var}}[X(t)].$$

Example

Orbit with uncertain initial speed $v_0 \pm \Delta v$: each run is a deterministic orbit, together they show how the uncertainty spreads.

Many deterministic simulations $\rightarrow$ a statistical description

Stochastic dynamics: Euler–Maruyama

$$dX = f(X, t) dt + g(X, t) dW_t, \quad dW_t \sim \mathcal{N}(0, dt).$$

The Euler–Maruyama update is Euler’s method plus a random kick:

$$X_{n+1} = X_n + f(X_n, t_n) \Delta t + g(X_n, t_n) \sqrt{\Delta t} \xi_n, \quad \xi_n \sim \mathcal{N}(0, 1).$$

Link to Part A

With $g = 0$ it reduces to Euler. Even with identical initial conditions, repeated runs give **different** trajectories. Its strong convergence order is only $1/2$, and stochastic methods need **their own convergence checks**: the $h/2$ test of Part A does not carry over directly.

One simulation versus an ensemble

A deterministic run gives $x(t)$. A stochastic run gives one *realisation* $X^{(i)}(t)$. Repeat N times:

$$\hat{\mathbb{E}}[X(t)] = \frac{1}{N} \sum_{i=1}^N X^{(i)}(t), \quad \widehat{\text{Var}}[X(t)] = \frac{1}{N-1} \sum_{i=1}^N \left(X^{(i)}(t) - \hat{\mathbb{E}}[X(t)] \right)^2.$$

The question changes

From “what trajectory occurs?” to “what is the distribution of trajectories?” The sampling error of the ensemble mean shrinks only as $1/\sqrt{N}$.

Numerical error is not physical uncertainty

Physical / modelling	Numerical
Measurement uncertainty	Discretisation error (finite h)
Parameter uncertainty	Round-off error (finite precision)
Initial-condition uncertainty	
Model uncertainty (neglected physics)	

Connection to Part A

Halving h reduces only the discretisation error. More numerical precision does **not** give more physical certainty.

A note on quantum systems

$$i\hbar \frac{\partial \psi}{\partial t} = \hat{H}\psi$$

The labels again

For an isolated system the Schrödinger evolution of ψ is linear and deterministic. Measurement outcomes are probabilistic, with probabilities given by $|\psi|^2$.

The same habit

What should be conserved? Here the norm, $\int |\psi|^2 dx = 1$. Check that your numerical solution preserves it, just as you checked the energy.

Linear, deterministic dynamics with a probabilistic description of outcomes: one more reminder that the labels are independent.

What should I expect, and how do I check?

System type	What to expect	Check with
Isolated, conservative	Constant E and ℓ	δ_E, δ_ℓ against h and method
Closed	Energy changes only via Q, W	Energy balance
Open	Steady state, sustained oscillation, or other driven behaviour	Steady state by root finding; time constant
Linear, dissipative	Decay; superposition	Exact solution; eigenvalues
Nonlinear	Limit cycles, several states, possible stiffness	Vary initial conditions; adaptive or stiff solver
Nonautonomous	Behaviour follows the time-varying parameters	Time-resolved analysis, not only a spread
Uncertain or stochastic	Spread of outcomes	Ensemble statistics; behaviour with N
Quantum (isolated)	Constant norm; probabilistic outcomes	$\int \psi ^2 dx = 1$

Verification and validation

Verification

Did we solve the model correctly?

- convergence tests ($h/2$ for a fixed step)
- conserved quantities
- exact solutions where they exist

Validation

Does the model represent the system?

- comparison with experimental data
- prediction of independent observations
- examining the modelling assumptions

Part A gave you the verification tools; Part B tells you what to compare with.

Before you simulate: six questions

The four classifications become a practical checklist:

1. What crosses the boundary? (*which balance or conserved quantity*)
2. Linear or nonlinear? (*exact solution? stiffness?*)
3. Static or dynamic; if dynamic, autonomous or not? (*root finding or time stepping? does the rule change in time?*)
4. Conservative or dissipative? (*what should the energy do?*)
5. Where, if anywhere, does randomness enter? (*inputs, measurements, dynamics*)
6. What do I expect, and how will I check it?

Then choose the method (Part A), and verify convergence.

TO TAKE AWAY

Hypothesis first, numerical methods second.

Know the system, expect its behaviour, choose the method, check the result.